

FITFLOW FITNESS TRACKER

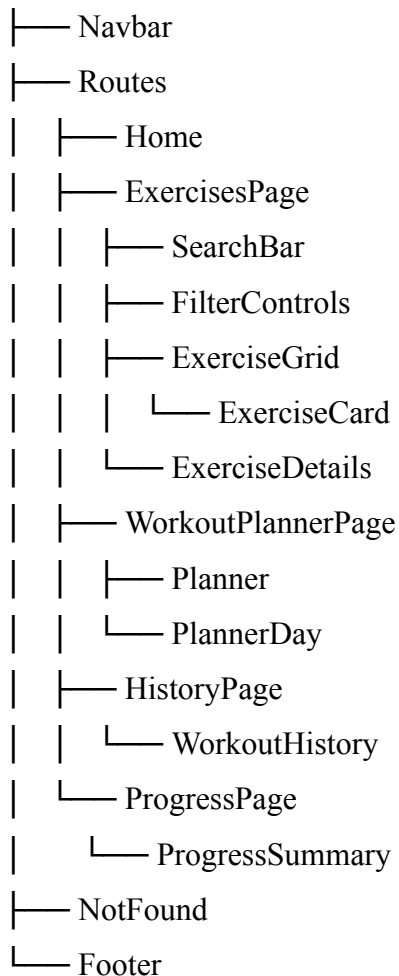
Project Planning Document

Project: React Fitness Tracker & Workout Planner

Technology: React, JavaScript, CSS, Jest, React Testing Library

1. COMPONENT HIERARCHY

App



FitnessProvider



2. DATA FLOW DIAGRAM

Exercise Data



ExercisesPage



Search / Filters



ExerciseGrid → ExerciseCard



ExerciseDetails

User Actions



FitnessContext



Planner / Workout History



localStorage



Updated UI

Exercise information is stored in the application data. User-generated planner and workout-history data is managed through FitnessContext and persisted using localStorage.

3. COMPONENTS TO BE CREATED

App, Navbar (rendered via a thin Header wrapper), Footer, Home, ExercisesPage, SearchBar, ExerciseDetail, WorkoutPlanner, HistoryPage, ProgressPage, NotFound, and FitnessProvider. Filtering, the exercise grid/cards, planner days, workout history, and progress summaries are

implemented inline within their parent pages rather than as separate components - there is no standalone FilterControls, ExerciseGrid, ExerciseCard, Planner, PlannerDay, WorkoutHistory, or ProgressSummary. A handful of shared UI components not in the original plan were also built: Button, Card, Badge, Modal, Loading, and the AudioPlayer/VideoPlayer pair used on the Home page.

4. PROPS FLOW

ExercisesPage computes the filtered exercise list internally with a useMemo and renders each result directly with .map(), passing individual exercise objects inline to the card markup — there is no separate ExerciseGrid or ExerciseCard component. Exercise information includes properties such as name, category, muscleGroups (an array), difficulty and image.

WorkoutPlannerPage holds the selected day and each day's exercise list in its own state and renders the day tabs and exercise list directly — there is no separate PlannerDay component. Shared planner and history functions (addToPlanner, removeFromPlanner, logWorkout) are provided through FitnessContext and consumed via the useFitness hook wherever they're needed..

5. STATE MANAGEMENT STRATEGY

useState manages temporary component state such as search terms, filters and selected exercises. FitnessContext and useContext manage shared planner and workout-history state. localStorage persists user data between sessions without requiring a backend. React Router manages page navigation.

6. TESTING STRATEGY

UI Components: Jest and React Testing Library test rendering, props, buttons, links and user interactions for Button, Card, Badge, Modal, SearchBar, Navbar, Footer, Loading, AudioPlayer, and VideoPlayer.

Pages: Tests verify required content, inline child markup, and important navigation for Home, ExercisesPage, ExerciseDetail, WorkoutPlanner, HistoryPage, ProgressPage, and NotFound.

FitnessContext: Tests cover initial state, localStorage loading and saving, adding/removing exercises, duplicate prevention, and workout history logging.

Filtering and Sorting: Since filtering and sorting logic lives inline in ExercisesPage (via useMemo) rather than in standalone utility functions, this behavior is verified through ExercisePage.test.jsx rather than a separate utility-function test suite.

Integration: Key user flows are tested end-to-end in WorkoutFlow.test.jsx, including navigating to the exercise library, searching for an exercise, adding it to the planner, and confirming the plan persists across navigation.

Test coverage is monitored using Jest's --coverage option.